

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development Practical
Practical – VII

Roll No.:T007	Name:subhashree jena
Class:TYIT	Batch:1
Date of Assignment:7/08/26	Date/Time of Submission:7/08/26

4. Implement k-fold cross-validation on a classification model.

CODE:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Create classification model
model = KNeighborsClassifier()
# Apply 5-fold cross-validation
scores = cross_val_score(model, X, y, cv=5)
print("Accuracy scores for each fold:")
print(scores)
print("Average Accuracy:", round(scores.mean() * 100, 2), "%")
print("Subhashree Jena T007")
```

OUTPUT:-

```
Accuracy scores for each fold:
[0.96666667 1.         0.93333333 0.96666667 1.         ]
Average Accuracy: 97.33 %
Subhashree Jena T007
```

Different Dataset:

supermarket_sales.csv

CODE:-

```
import pandas as pd
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder
df = pd.read_csv("supermarket_sales.csv")
df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')
df = df.drop('invoice_id', axis=1, errors='ignore')
le = LabelEncoder()
categorical_cols = ['branch', 'city', 'customer_type', 'gender',
                    'product_line', 'payment']

for col in categorical_cols:
    if col in df.columns:
        df[col] = le.fit_transform(df[col])
X = df.drop('rating', axis=1)
y = (df['rating'] >= 7).astype(int)
model = KNeighborsClassifier()
scores = cross_val_score(model, X, y, cv=5)
print("Accuracy scores for each fold:")
print(scores)
print("Average Accuracy:", round(scores.mean() * 100, 2), "%")
print("Subhashree Jena T007")
```

OUTPUT:-

```
Accuracy scores for each fold:  
[0.75  0.6  0.65  0.7  0.625]  
Average Accuracy: 66.5 %  
Subhashree Jena T007
```

CONCLUSION:-

k-fold cross-validation on a classification model using both the Iris dataset and the supermarket_sales dataset. This method helped in evaluating the model's performance more reliably by dividing the dataset into multiple folds and testing it on different subsets. It reduced the chances of overfitting and provided a better estimate of model accuracy. The results showed that simpler datasets like Iris give more consistent accuracy, whereas real-world datasets like supermarket_sales may produce varying results due to complexity and noise.

5. Implement a Decision Tree classifier using the Iris or any relevant dataset.

CODE:-

```
from sklearn.datasets import load_iris  
from sklearn.model_selection import train_test_split  
from sklearn.tree import DecisionTreeClassifier  
# Load dataset  
iris = load_iris()  
X = iris.data  
y = iris.target  
# Split dataset  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.3,  
    random_state=1)  
# Create model  
model = DecisionTreeClassifier()  
# Train model  
model.fit(X_train, y_train)  
# Display accuracy  
print("Training Accuracy:", model.score(X_train, y_train) * 100)  
print("Testing Accuracy:", model.score(X_test, y_test) * 100)  
# Prediction  
prediction = model.predict([X_test[0]])  
print("Predicted class:", prediction[0])  
print("Subhashree Jena T007")
```

OUTPUT:-

```
Training Accuracy: 100.0  
Testing Accuracy: 95.55555555555556  
Predicted Class: 0  
Subhashree Jena T007
```

Different Dataset:

titanic_sample.csv

CODE:-

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
import pandas as pd
# Load dataset
df = pd.read_csv("titanic_sample.csv")
# Preprocessing
df = df.drop(['PassengerId', 'Name', 'Ticket', 'Cabin'], axis=1)
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df['Embarked'] = df['Embarked'].map({'S': 0, 'C': 1, 'Q': 2})
# Features and target
X = df.drop('Survived', axis=1)
y = df['Survived']
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1)
# Create model
model = DecisionTreeClassifier()
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)
# Prediction
prediction = model.predict(X_test.iloc[0:1])
print("Predicted class:", prediction[0])
print("Subhashree Jena T007")
```

OUTPUT:-

```
Training Accuracy: 100.0
Testing Accuracy: 33.33333333333333
Predicted Class: 1
Subhashree Jena T007
```

CONCLUSION:-

implemented a Decision Tree classifier using the Iris dataset and the titanic_sample dataset. The model was trained and tested to classify outputs based on given features. It was observed that the Decision Tree performs very well on clean and simple datasets like Iris, giving high and stable accuracy. However, when applied to real-world datasets like Titanic, the accuracy and predictions may vary due to missing values, data preprocessing, and complexity. This practical helped in understanding how decision trees work and how dataset characteristics affect model performance.